BBVA
Creating Opportunities

Arquitectura Técnica de Desarrollo y Testing DMT

Guía de arquitectura, test data y patrones de reutilización

DMT



Documento técnico de referencia · Octubre 2026

Guía de Arquitectura de Desarrollo y Testing DMT

Referencia para onboarding, creación de clases test, mantenimiento, refactorización, diseño de test data y revisión de PRs

> **Principio central. El test expresa el caso que quiere validar; DataFactory representa escenarios reutilizables; las utilities DMT construyen objetos. Cuando corresponde, DMT puede consumir una base común externa. La reutilización nunca debe cambiar la semántica del test.**

Cómo leer esta guía

La guía aplica tanto a clases test nuevas como a refactorizaciones. En ambos casos el orden es el mismo: primero revisar si DMT_Test_DataFactory ya representa la casuística; después clasificar el ownership del objeto; finalmente elegir el método build/create y el patrón de DML adecuados. En un test nuevo evita crear test data manual antes de completar esta revisión; en un refactor, además, preserva la semántica y el patrón DML existentes.

Paso	Pregunta	Resultado esperado
1	¿Existe un generateX() que representa esta casuística?	DataFactory antes que reconstrucción manual.
2	Si no existe, ¿DMT especializa el objeto o DES ya lo cubre?	Seleccionar DMT_UTIL o, si es una entidad común ya soportada, consumir DES_UTIL como dependencia externa.
3	¿El registro se inserta inmediatamente o debe permanecer en memoria/bulk?	Elegir createX o buildX sin alterar DML.
4	¿Qué cambia en este escenario?	Expresarlo mediante overrides; no ampliar firmas por campos configurables.

Mapa del documento

Bloque	Contenido
Arquitectura	Ownership DMT y frontera con dependencias externas.
Decisión	Factory-first, ownership DMT, dependencia externa cuando aplique y build vs create.
Factory	Qué debe permanecer en DMT_Test_DataFactory y qué debe salir.
Métodos	Overrides, firmas, RecordTypeId, parámetros null y evolución segura.
Semántica	DML, bulk, @TestSetup, queries y side effects.
Aplicación	Ejemplos para tests nuevos/refactors, anti-patrones, checklist y prompt de apoyo.

1. Objetivo y principios de diseño

La arquitectura no persigue solo reducir líneas de código: busca que la creación de test data sea predecible, reutilizable y compatible con la semántica real de cada test.

DMT dispone de sus propias capas de test data y puede consumir una factoría compartida DES cuando necesita una entidad común ya soportada. DES se trata en esta guía únicamente como dependencia externa: no forma parte del alcance funcional ni de las decisiones de implementación del equipo DMT. DMT_Test_DataFactory actúa como capa de escenarios reutilizables del dominio DMT.

- Reutilizar antes de duplicar: primero escenarios existentes, después métodos build/create del objeto.
- Una sola fuente para defaults reutilizables; los valores del caso de prueba permanecen como overrides.
- Separar construcción y persistencia mediante el patrón Build/Create.
- No degradar bulk ni alterar DML/SOQL/side effects para “limpiar” un test.
- Tratar DES únicamente como dependencia externa: reutilizar los métodos públicos ya existentes de DES_UtilTestClass cuando corresponda y no modificar clases DES desde DMT.
- No inventar firmas: un método existe solo si su declaración real está localizada en código.

> **Contexto externo. DMT sigue el patrón Build/Create y puede consumir DES cuando necesita una entidad común ya soportada. DES se considera una dependencia compartida externa; esta guía no define ni gobierna su implementación interna.**

1.1. Alcance: tests nuevos, mantenimiento y refactorización

Esta guía es el estándar DMT para cualquier trabajo sobre clases test: creación de tests nuevos, ampliación de cobertura, mantenimiento y refactorización. En un test nuevo, la arquitectura debe aplicarse antes de escribir construcción manual de SObjects. En un refactor, primero se inventariará la creación existente y se preservan comportamiento, relaciones, DML/SOQL y side effects. En ambos casos la prioridad es la misma: escenario reutilizable en DMT_Test_DataFactory, después los métodos públicos de DMT_UtilTestClass y, solo cuando corresponda, el consumo de una dependencia externa compartida.

1.2. Tres categorías de objetos

Categoría	Quién construye	Regla
Objeto propio DMT	DMT_UtilTestClass / DMT_UtilTestClassBuild	DMT es propietario del builder y de sus defaults reutilizables.
Objeto compartido que DMT especializa	DES base → DMT Build	DES aporta estructura/defaults comunes; DMT añade solo sus mínimos.
Entidad común ya soportada por la dependencia externa	DES_UtilTestClass	DMT reutiliza DES_UtilTestClass como dependencia externa; no crea wrappers equivalentes ni modifica clases DES.

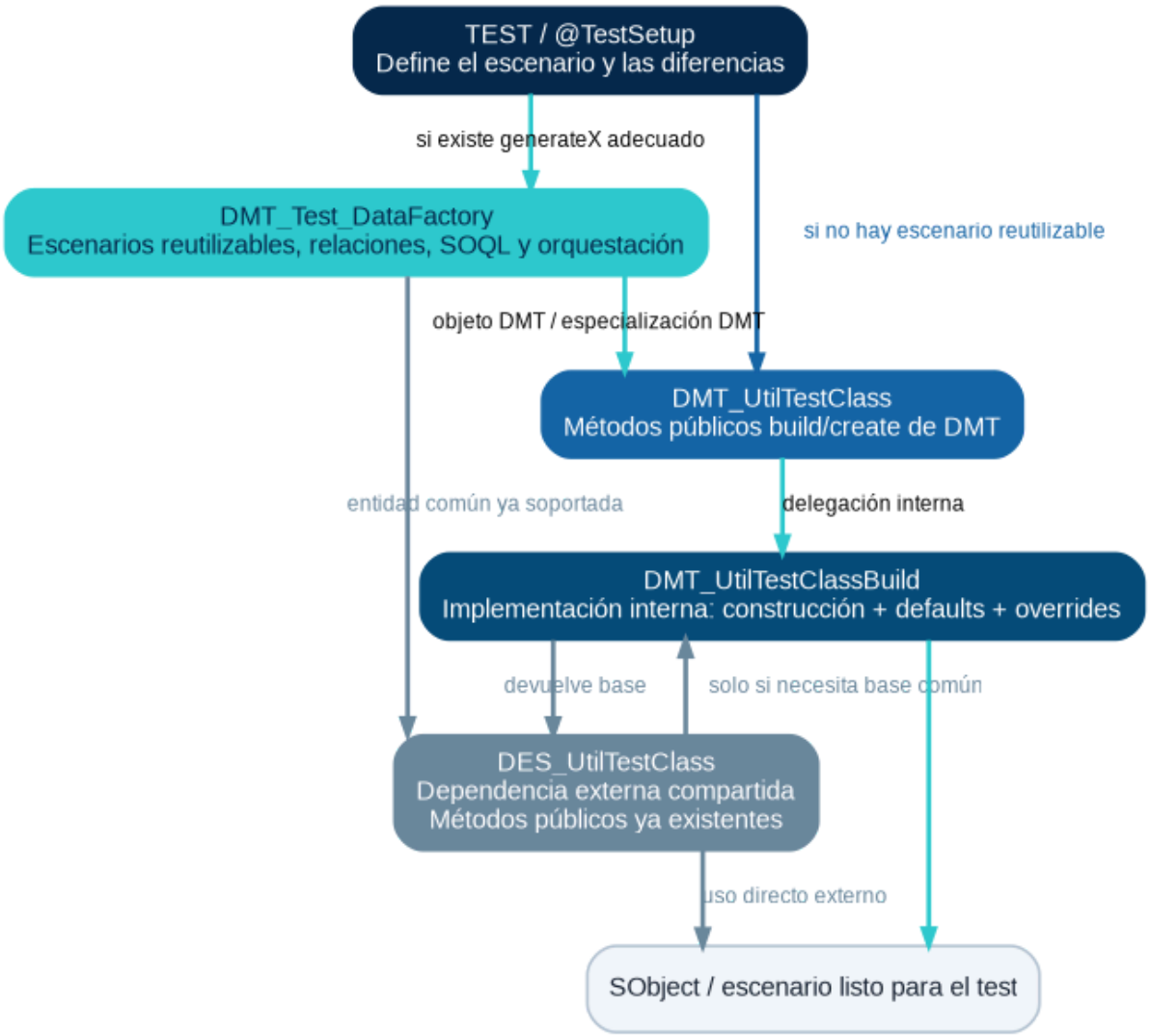


Figura 1. Flujo de dependencias DMT. DES aparece únicamente como dependencia externa cuando DMT necesita una base común.

2. Responsabilidades por clase

Cada clase DMT tiene una frontera clara. Cuando una responsabilidad cruza de capa, aumenta el acoplamiento y tanto el desarrollo nuevo como los refactors dejan de ser locales.

2.1. DMT_Test_DataFactory — escenarios y composición

Es la primera clase que debe revisarse cuando el test necesita datos. Su unidad de reutilización no es “un SObject”, sino una casuística reconocible del dominio: una Treasury Line completa, una estructura de aprobación, un conjunto de relaciones o cualquier escenario que tenga sentido reutilizar como bloque.

- Puede crear un único registro si ese registro representa un escenario reutilizable con semántica propia.
- Puede componer varios objetos, relaciones y dependencias entre Ids.
- Puede realizar SOQL, FOR UPDATE y orquestación cuando forman parte de la preparación del escenario.
- Debe preservar el patrón DML y los side effects del escenario que representa.
- Usa DMT_UTIL para objetos DMT o especializados por DMT; si una entidad común ya está soportada externamente, consume DES_UTIL sin crear un wrapper DMT.
- Los valores específicos del escenario se expresan como overrides; no se convierten automáticamente en defaults globales.

2.2. DMT_UtilTestClass — fachada pública de objeto

Expone los métodos canónicos build/create cuando DMT es responsable del objeto o añade comportamiento real. No construye Maps estructurales ni duplica inicialización: los build delegan a DMT_UtilTestClassBuild y los create realizan DML explícito sobre el build correspondiente.

Patrón de fachada

```
public Case buildCase(Id ownerId, Map<Schema.SObjectField, Object> overrides) {
    return DMT_UtilTestClassBuild.buildCase(ownerId, overrides);
}

public Case createCaseRecord(Id ownerId, Map<Schema.SObjectField, Object> overrides) {
    return (Case) insertAndReturn(buildCase(ownerId, overrides));
}
```

2.3. DMT_UtilTestClassBuild — construcción memory-only

Contiene el detalle de construcción y no debe invocarse directamente desde tests ni desde DMT_Test_DataFactory. No ejecuta DML ni SOQL. Para objetos compartidos puede partir de DES_UtilTestClass; para objetos DMT crea el SObject directamente. Los overrides siempre se aplican al final. Regla estricta: un test o DataFactory nunca llama directamente a DMT_UtilTestClassBuild ni a DES_UtilTestClassBuild; siempre utiliza la clase Util correspondiente. No confundir la clase Build con el método buildX(): buildX() sí es correcto cuando se necesita construcción en memoria, modificación previa al insert o preparación de varios registros para un único DML.

Builder canónico con overrides

```
public static DMT_Feature__c buildFeature() {
    return new DMT_Feature__c(gf_audit_date__c = Date.today());
}

public static DMT_Feature__c buildFeature(Map<Schema.SObjectField, Object> overrides) {
    return (DMT_Feature__c) applyOverrides(buildFeature(), overrides);
}
```

2.4. DMT_UtilTestClassConstants — defaults reutilizables

Centraliza valores de test realmente comunes al dominio DMT: defaults, developer names, Record Types reutilizables u otros valores estables. No es un almacén de datos específicos de escenarios.

2.5. Dependencia externa: DES_UtilTestClass

DES_UtilTestClass se menciona únicamente como dependencia compartida externa. DMT puede reutilizar sus métodos públicos cuando una entidad común ya está soportada, o utilizarla como base desde DMT_UtilTestClassBuild. Este documento no describe ni gobierna la implementación del equipo DES. Para resolver necesidades DMT no se modifican clases DES.

Alcance de esta guía: DMT. La referencia a DES sirve exclusivamente para saber cuándo consumir una capacidad externa y evitar duplicaciones. Las reglas, ejemplos y extensiones descritas aquí gobiernan únicamente las capas DMT.

3. Qué debe mantenerse en DMT_Test_DataFactory

Esta es la frontera más importante para evitar dos extremos: tests que reconstruyen escenarios completos y utilities que absorben lógica de negocio o DML que no les corresponde.

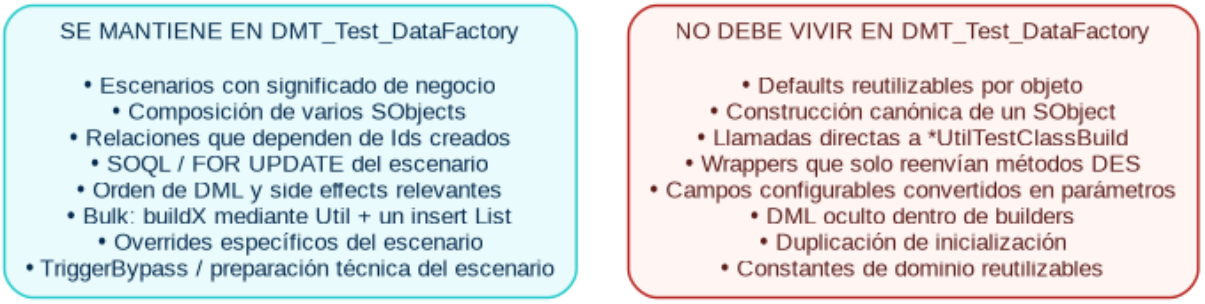


Figura 2. Frontera práctica: escenario/orquestación en Factory; construcción/defaults en Util/Build/Constants.

3.1. Criterio de permanencia en Factory

Un bloque debe permanecer en DMT_Test_DataFactory cuando su valor de reutilización está en la combinación de registros y en la secuencia del escenario, no en la construcción aislada de un SObject.

Mantener en Factory cuando…	Ejemplo
El escenario crea varios objetos relacionados y necesita sus Ids en pasos posteriores.	Account → Line → RiskLineTerm → Product/Feature.
El orden de insert/query/update forma parte de la preparación.	Insert, query FOR UPDATE y creación de hijos.
El escenario necesita un conjunto estable de overrides de negocio.	Treasury Line con estado, país, límites y productos concretos.
Debe conservarse un único insert List para bulk.	buildX() de N registros + insert lista.
La preparación incluye TriggerBypass, custom settings o side effects deliberados.	Activar bypass, ejecutar DML y restaurar contexto según el escenario.
Existe un nombre de dominio útil y reutilizable para el escenario.	generateTestTreasuryLines(), generateApprovalScenario(...).

3.2. Qué no debe desplazarse a Factory

- Defaults de un objeto que todos los tests DMT deberían recibir.
- La construcción canónica de un SObject individual.
- Wrappers DMT que solo reenvían un método ya existente en DES_UtilTestClass sin añadir comportamiento.
- Nuevos parámetros por cada campo configurable; para eso existen overrides.
- Lógica de applyOverrides, validación de campos o detalles internos del builder.
- DML oculto que haga imposible saber desde el caller cuándo se persiste el registro.

Factory-first no significa Factory-always. Un generateX() se usa solo si reproduce la misma casuística y conserva DML/bulk/side effects. Si un generateX() hace inserts individuales y el test necesita un único insert List, el builder sigue siendo la elección correcta.

4. Árbol de decisión para crear o refactorizar un test

La secuencia de decisión debe ser consistente. Evita decidir “build vs create” antes de haber comprobado si el escenario ya existe en DataFactory.

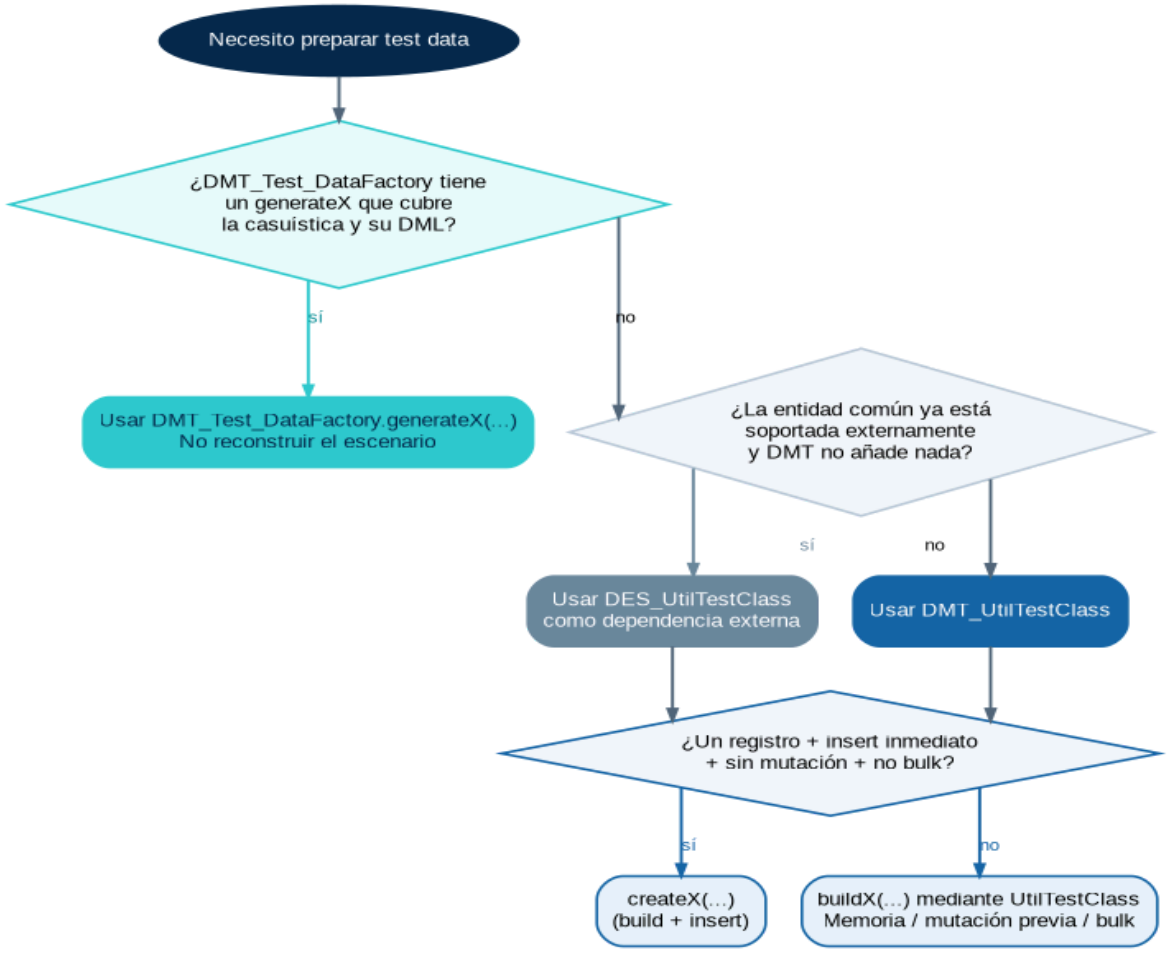


Figura 3. Decisión Factory-first → ownership DMT → dependencia externa cuando corresponda → build/create.

4.1. Regla operativa

Orden	Decisión	Qué comprobar
1	Buscar generateX() relacionado	Firma real, registros que crea, DML, queries y side effects.
2	Clasificar ownership	DMT puro / DMT sobre una base compartida / entidad externa ya cubierta.
3	Elegir método	DMT_UTIL o, cuando corresponda, DES_UTIL como dependencia externa; nunca las clases *UtilTestClassBuild directamente desde el test o DataFactory.
4	Elegir build/create	Semántica de memoria, mutación, insert inmediato y bulk.
5	Aplicar overrides	Solo diferencias del escenario; defaults quedan en la capa responsable.
6	Auditar resultado	Mismo comportamiento, DML y relaciones; sin nuevos wrappers ni null placeholders.

4.2. Cuándo usar createX

Usa createX cuando el caller necesita un único registro persistido inmediatamente, no lo modifica antes del insert y no forma parte de una operación bulk que deba conservarse.

Registro único con insert inmediato

```
Account acc = DMT_UTIL.createAccount(
    new Map<Schema.SObjectField, Object>{
        Account.CIB_Commercial_Prospect__c => 'Commercial'
    }
);
```

4.3. Cuándo usar buildX

Usa buildX cuando el registro solo debe existir en memoria, necesita mutaciones antes de insertarse o se acumula con otros registros para un único DML.

Bulk: build + insert List

```
List<DMT_Feature__c> features = new List<DMT_Feature__c>{
    DMT_UTIL.buildFeature(overridesA),
    DMT_UTIL.buildFeature(overridesB)
};
insert features; // un único DML
```

5. Overrides y evolución segura de métodos y firmas

Los overrides son el mecanismo normal para expresar variaciones de escenario. Una firma pública no debe crecer por cada campo configurable.

5.1. Prioridad antes de cambiar una firma

Prioridad	Opción	Uso
1	Método existente + overrides	Preferida para campos configurables y diferencias de escenario.
2	Nuevo overload	Solo si aporta una forma de uso realmente distinta y estable.
3	Nuevo argumento	Solo para dependencia estructural genuina que no debe tratarse como campo configurable.
4	Nuevo método	Último recurso; evita nombres alternativos para el mismo SObject.

5.2. RecordTypeId debe ser override cuando es configuración del escenario

El Record Type suele variar entre tests y escenarios; por tanto no debe convertirse en parámetro posicional de los builders DMT salvo que exista una razón estructural excepcional y explícita. En los builders normalizados se envía mediante overrides.

Correcto: RecordTypeId como override

```
Case approvalCase = DMT_UTIL.createCaseRecord(
    ownerId,
    new Map<Schema.SObjectField, Object>{
        Case.RecordTypeId => approvalCaseRecordTypeId,
        Case.Status => 'Pending',
        Case.Subject => 'Test'
    }
);
```

5.3. No utilizar null para mantener firmas antiguas

Si la clase actual ofrece buildX() y buildX(overrides), no se deben mantener llamadas del tipo buildX(null, null) para simular argumentos antiguos. Las referencias deben migrarse a la firma canónica actual.

Evitar	Usar
DMT_UTIL.buildTaskStatusWhenCompleted(null, null)	DMT_UTIL.buildTaskStatusWhenCompleted()
DMT_UTIL.buildTaskStatusWhenCompleted(null, null, overrides)	DMT_UTIL.buildTaskStatusWhenCompleted(overrides)

5.4. Builder sin parámetros cuando los campos pertenecen al escenario

Ejemplo: DMT_Line__c

```
public static DMT_Line__c buildLine() {
    return new DMT_Line__c();
}

public static DMT_Line__c buildLine(Map<Schema.SObjectField, Object> overrides) {
    return (DMT_Line__c) applyOverrides(buildLine(), overrides);
}
```

6. Defaults, constantes y ownership de datos

La ubicación de un valor depende de quién debe controlarlo, no de cuántas veces aparece en un fichero.

Tipo de valor	Ubicación	Ejemplo conceptual
Default compartido aportado por DES externo	Gestionado externamente por DES; DMT no lo redefine	Base común consumida por DMT cuando corresponde.
Default DMT reutilizable	DMT_UtilTestClassConstants / DMT Build	Currency, flags o nombres comunes de DMT.
Valor específico del escenario	DataFactory/test override	Status, Line_Id, RecordTypeId o relación concreta del caso.
Valor técnico temporal	Test/helper local	Datos usados solo para simular old/new, payloads o inputs del método bajo test.

No promover por repetición. Que un literal aparezca varias veces no implica que sea un default global. Si cambia entre escenarios o describe un caso de negocio concreto, debe permanecer como override de escenario.

6.1. Integración DMT con una base común externa

Cuando DMT necesita una entidad común ya soportada, puede reutilizar los métodos públicos de DES_UtilTestClass como base y añadir únicamente los mínimos DMT. Este patrón documenta únicamente cómo DMT consume esa dependencia; su implementación queda fuera del alcance de esta guía.

```
public static Account buildAccount() {
    Account acc = DES_UTIL.buildAccount();
    return (Account) applyOverrides(
        acc,
        new Map<Schema.SObjectField, Object>{
            Account.Advising__c => DMT_UtilTestClassConstants.DEFAULT_ACCOUNT_ADVISING,
            Account.CurrencyIsoCode => DMT_UtilTestClassConstants.DEFAULT_CURRENCY
        }
    );
}

public static Account buildAccount(Map<Schema.SObjectField, Object> overrides) {
    return (Account) applyOverrides(buildAccount(), overrides);
}
```

Precedencia. DES defaults < DMT defaults < overrides explícitos del caller.

7. DML, bulk y semántica del test

La creación o refactorización de test data es correcta cuando utiliza la capa adecuada sin alterar la semántica del test. En código existente deben preservarse DML, queries y side effects relevantes; en tests nuevos deben diseñarse desde el principio con la mínima cantidad de DML y sin duplicar construcción ya disponible.

7.1. Contrato Build/Create

Método	Estado al devolver	DML	Uso principal
buildX(...)	Objeto en memoria	0	Memoria, mutación previa, listas bulk.
createX(...)	Objeto persistido	1 insert	Registro único con persistencia inmediata.
generateX(...)	Escenario definido por Factory	Variable	Debe verificarse su comportamiento real antes de usarlo.

7.2. Bulk no se degrada

Correcto: construir N registros y persistirlos juntos

```
List<Case> casesToInsert = new List<Case>();
for (ScenarioInput input : inputs) {
    casesToInsert.add(
        DMT_UTIL.buildCase(
            input.ownerId,
            new Map<Schema.SObjectField, Object>{
                Case.RecordTypeId => input.recordTypeId,
                Case.Subject => input.subject
            }
        )
    );
}
insert casesToInsert; // conservar el único DML
```

- No reemplazar un insert List por N createX().
- No introducir DML dentro de loops.
- Preservar orden relevante entre DML y SOQL.
- Preservar FOR UPDATE, queries posteriores, TriggerBypass y side effects.
- Preservar Test.startTest()/Test.stopTest() y los asserts.

7.3. @TestSetup: compartir fixtures, no escenarios

Buen candidato para @TestSetup	Debe permanecer en cada @IsTest
Account/Opportunity base común y determinista.	Datos exclusivos del escenario o que cambian por test.
Relaciones base repetidas por varios tests.	DML cuyo momento forma parte del comportamiento validado.
Fixtures que no son la acción probada.	Registros cuya compartición reduce aislamiento o enmascara la intención.

8. Ejemplos para tests nuevos y refactorizaciones

Los ejemplos siguientes muestran cómo aplicar las reglas sin perder intención ni comportamiento.

8.1. Caso + RecordTypeId

Campo configurable → override

```
// Antes
Case c = DMT_UTIL.createCaseRecord(ownerId, recordTypeId,
    new Map<Schema.SObjectField, Object>{
        Case.Status => 'Pending'
    }
);

// Después
Case c = DMT_UTIL.createCaseRecord(ownerId,
    new Map<Schema.SObjectField, Object>{
        Case.RecordTypeId => recordTypeId,
        Case.Status => 'Pending'
    }
);
```

8.2. Escenario Treasury Line

Si DMT_Test_DataFactory ya contiene un generateTestTreasuryLines() que crea la composición necesaria y su DML coincide con lo que el test necesita, se reutiliza ese escenario. Si el test solo necesita una Line aislada o un patrón bulk distinto, se baja a DMT_UTIL.buildLine(overrides) sin forzar el generateX().

8.3. Feature en bulk

No convertir a dos createFeatureRecord()

```
List<DMT_Feature__c> featureList = new List<DMT_Feature__c>{
    DMT_UTIL.buildFeature(featureOverridesA),
    DMT_UTIL.buildFeature(featureOverridesB)
};
insert featureList;
```

8.4. Entidad común ya cubierta por una dependencia externa

Si Product2 ya está soportado por la dependencia compartida y DMT no necesita comportamiento adicional, el test/DataFactory utiliza el método real disponible en DES_UtilTestClass. No se crea DMT_UTIL.buildProduct() solo para renombrar o reenviar.

9. Anti-patrones y señales de revisión

Estas señales suelen indicar que la responsabilidad está en la capa equivocada o que se están añadiendo métodos o parámetros sin necesidad.

Señal	Problema	Corrección
buildX(null, null, overrides)	Firma antigua conservada mediante placeholders.	Usar buildX(overrides) si es la firma canónica actual.
RecordTypeId como argumento posicional configurable	Crecimiento innecesario de firmas.	Mover a overrides.
new SObject(...) repetido en tests	Construcción distribuida y defaults inconsistentes.	Buscar DataFactory/Util existente antes de mantenerlo.
Wrapper DMT idéntico a un método DES existente	Duplicación de ownership.	Consumir DES_UTIL como dependencia externa.
createX dentro de un loop	DML multiplicado.	buildX + insert List.
DMT_UtilTestClassBuild desde un test	Bypass de fachada.	Usar DMT_UtilTestClass.
DES_UtilTestClassBuild desde DMT	Dependencia con implementación interna DES.	Usar DES_UtilTestClass.
Map merged en la fachada para traducir parámetros	La fachada empieza a construir objetos.	Builder/overrides; fachada solo delega.

Excepción técnica válida. No toda construcción manual es test data reutilizable. List/Map/Set auxiliares, payloads, objetos usados para simular Trigger.old/oldMap o clone(true) pueden permanecer locales si su propósito es técnico y está claro.

10. Cómo extender la arquitectura

Cuando no exista soporte real, la solución debe añadirse en la capa responsable y con el mínimo cambio de métodos o firmas.

Necesidad	Componente	Responsabilidad
Default DMT reutilizable	DMT_UtilTestClassConstants	Centralizar un valor estable de dominio.
Construcción de objeto DMT	DMT_UtilTestClassBuild	Crear en memoria y aplicar defaults/overrides.
Especialización DMT de una base externa DES	DMT_UtilTestClassBuild	Consumir DES_UTIL como base externa y añadir únicamente mínimos DMT.
Métodos públicos build/create	DMT_UtilTestClass	Delegar build; create = build + insert.
Escenario reutilizable	DMT_Test_DataFactory	Componer registros, relaciones, DML/SOQL y overrides del escenario.

10.1. Antes de añadir un método

- Confirmar que DataFactory no cubre ya la casuística.
- Confirmar que una Método existente + overrides no es suficiente.
- Confirmar que DMT Util no tiene ya el builder/create canónico.
- Para objetos comunes, confirmar que DES no cubre exactamente la necesidad.
- Si se añade una firma pública, evaluar y actualizar todas sus referencias.
- No añadir compatibilidad legacy por defecto: actualizar las referencias cuando las firmas se están normalizando de forma deliberada.

10.2. Propuesta mínima cuando falta soporte

```
SObject:
Necesidad / escenario:
DataFactory revisado:
Métodos DMT reales revisados:
Dependencia externa revisada (si aplica):

Capa propuesta:
Método / firma:
¿Consume una base externa?:
Defaults DMT:
Overrides del escenario:
DML / bulk:
Impacto en referencias:
Ejemplo de uso final:
```

11. Workflow recomendado para crear o refactorizar una clase test

El objetivo es convertir el análisis en una secuencia repetible y revisable en PR.

Fase	Acción	Salida
Inventario	Localizar new SObject, insert, listas, helpers y métodos que devuelven registros.	Lista de SObjects y patrón DML actual.
Factory-first	Buscar generateX relacionados y comprobar su comportamiento real.	Usar / descartar con motivo.
Ownership	Clasificar objeto DMT, objeto común especializado por DMT o entidad común ya soportada externamente.	Capa correcta por objeto.
Método	Localizar firmas reales build/create y overloads con overrides.	No inventar llamadas.
Migración	Sustituir creación manual preservando DML, relaciones y datos.	Código equivalente y más reutilizable.
Normalización	Eliminar null placeholders / parámetros configurables obsoletos y actualizar referencias.	Firmas coherentes.
Auditoría	Buscar new SObject restantes y referencias a firmas antiguas.	Pendientes justificados o propuesta.

11.1. Auditoría de creación manual

```
Buscar globalmente:
new Account(...)
new Case(...)
new Opportunity(...)
new DMT_*__c(...)
insert new X(...)
list.add(new X(...))
return new X(...)

Y también llamadas obsoletas:
buildX(null
createX(null
, null
null, null
```

Criterio de salida. No debe quedar un new SObject(...) de test data sin haber comprobado si existe un método reutilizable en DMT_Test_DataFactory o en la clase Util correspondiente, o sin una justificación técnica concreta.

12. Checklist para desarrollo y revisión de PR

Esta lista puede utilizarse tanto por el autor como por el reviewer. En tests nuevos valida que la arquitectura se aplique desde el inicio; en refactors valida además que se conserve la semántica y el patrón DML existentes.

Check	Pregunta
☐	¿Se revisó DMT_Test_DataFactory antes de reconstruir escenarios?
☐	¿Los generateX descartados tienen un motivo técnico (DML, side effects, casuística distinta)?
☐	¿Cada objeto está clasificado como Objeto DMT, objeto común especializado por DMT o entidad común ya soportada externamente?
☐	¿Ningún test ni DMT_Test_DataFactory invoca directamente DMT_UtilTestClassBuild o DES_UtilTestClassBuild?
☐	¿buildX es memory-only y createX hace DML explícito?
☐	¿Los overrides expresan los campos del escenario y ganan sobre defaults?
☐	¿RecordTypeId y otros campos configurables se pasan por overrides cuando corresponde?
☐	¿Se eliminaron placeholders null de firmas antiguas?
☐	¿No se degradó un insert List a múltiples inserts individuales?
☐	¿No hay DML en loops?
☐	¿Se preservaron SOQL/FOR UPDATE/TriggerBypass/startTest-stopTest/side effects relevantes?
☐	¿Los defaults reutilizables están en Constants/Build y los valores de escenario en Factory/test?
☐	¿¿No se creó un wrapper DMT para una capacidad que ya cubre la dependencia externa DES?
☐	¿Cualquier new SObject restante está justificado?

13. Prompt corto de apoyo para Copilot

El prompt no sustituye la revisión técnica, pero obliga al asistente a seguir el orden de decisión de la arquitectura.

```
Crea o refactoriza esta clase test siguiendo la arquitectura DMT existente. DES es solo una dependencia externa de referencia/consumo cuando una entidad común ya está soportada.

1) Revisa PRIMERO DMT_Test_DataFactory. Para cada escenario, localiza generateX() reales y úsalo solo si reproduce la misma casuística y conserva DML/bulk/side effects.
2) Si Factory no cubre el caso, clasifica el objeto: Objeto DMT / objeto común especializado por DMT / entidad común ya soportada externamente. Usa DMT_UtilTestClass o DES_UtilTestClass; nunca llames directamente a DMT_UtilTestClassBuild ni DES_UtilTestClassBuild. buildX() sí se utiliza, pero siempre a través de la clase Util correspondiente.
3) Usa createX solo para un único registro con insert inmediato, sin mutación previa y fuera de bulk. Usa buildX para memoria, mutación previa o build de varios registros + un único insert List.
4) Mantén valores específicos como Map<Schema.SObjectField,Object> overrides. No repitas defaults de la utility, no pases null como placeholder y no añadas RecordTypeId u otros campos configurables como argumentos si pueden ir en overrides.
5) Preserva asserts, relaciones, orden y agrupación de DML, SOQL/FOR UPDATE, TriggerBypass, side effects y Test.startTest()/stopTest(). No introduzcas DML en loops.
6) No inventes métodos ni firmas. Si falta soporte, mantén el código funcional y propón la extensión mínima indicando la capa correcta.

Al terminar indica: generateX revisados/usados/descartados, métodos DMT utilizados y dependencias externas consumidas, elección build/create, creaciones manuales restantes y confirmación de que bulk y comportamiento se preservan.
```

13.1. Resumen para recordar

> **Regla de oro.** Factory representa escenarios; Util expone los métodos build/create; Build construye internamente; Constants centraliza defaults; una dependencia compartida externa puede aportar una base común; el test expresa solo lo específico.

Referencias de diseño

- Arquitectura Técnica de Desarrollo y Testing DMT — documento base actualizado para esta guía.
- Guía de uso de Test Data Factories — referencia corporativa del patrón Build/Create y separación Util / Build / Constants. Se usa como contexto, no como documentación del equipo DES.
- Implementación actual de DMT_UtilTestClass, DMT_UtilTestClassBuild y DMT_Test_DataFactory como fuente de verdad para firmas reales.